



Domain Specific Language Design (2IMP20): Syntax



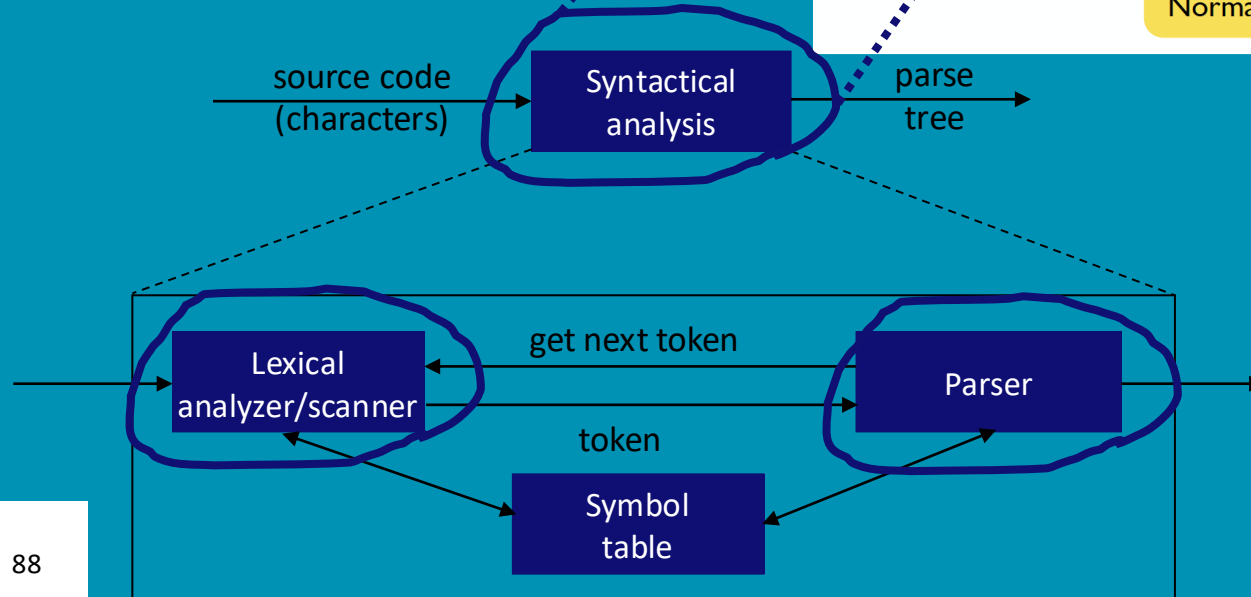
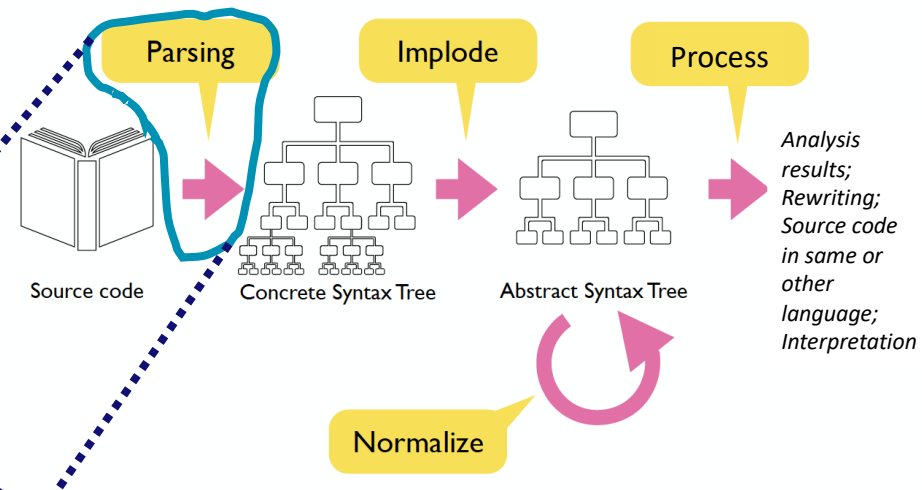
Loek Cleophas



Ivan Kurtev

Lexical and syntactical analysis using LEX and YACC

Recall: Scanning/lexical analysis + parsing/syntactical analysis



... but why this old stuff??

- FORTRAN,
- COBOL,
- LEX and YACC

*instead of DSLs and
Rascal/EMF/Xtext/
ANTLR/Langium...*



The screenshot shows a news article from NRC. The header includes the NRC logo and navigation links for 'Mijn nieuws', 'Podcasts', 'Digitale krant', a search icon, and a user profile icon. The article is categorized as 'ACHTERGROND'. The title is 'Gezocht: mensen die een programmeertaal uit 1959 willen leren (want die heeft nog altijd toekomst)'. The text mentions that large organizations like the tax authority and UWV are looking for people who can use COBOL, an old programming language. It notes that COBOL training projects (with a guarantee) must offer an outcome, and that working with COBOL gives an adrenaline rush, similar to a top sport. The author is Timo Nijssen, dated May 1, 2024, with a 4-minute read time. At the bottom, there are buttons for 'Luisteren' (Listen) and 'Leeslijst' (Reading list).

nrc

Mijn nieuws Podcasts Digitale krant

■ ACHTERGROND

Gezocht: mensen die een programmeertaal uit 1959 willen leren (want die heeft nog altijd toekomst)

Cobol Grote instellingen als de Belastingdienst en het UWV hebben een prangend tekort aan programmeurs die overweg kunnen met de stokoude taal Cobol. Omscholingstrajecten (met baangarantie) moeten een uitkomst bieden. „Werken met Cobol geeft me adrenaline, zoals ik gewend was uit de topsport.”

Timo Nijssen • 1 mei 2024 • Leestijd 4 minuten

Luisteren 🎧 Leeslijst 📖

(source: <https://www.nrc.nl/nieuws/2024/05/01/gezocht...>)

Lexical and syntactical analysis using LEX and YACC

- YACC: Yet Another Compiler Compiler
- LEX and YACC work as a team
- Old: 1960s/1970s...
- *originally...*
- *extensively used*
- *highly available*
 - **lex**, **yacc** on most UNIX systems
 - **bison**: a yacc replacement from GNU
 - **flex**: fast lexical analyzer
 - BSD yacc
 - Versions for other OS'es (MS Windows, Linux, ...)

Lexical analysis using (F)LEX

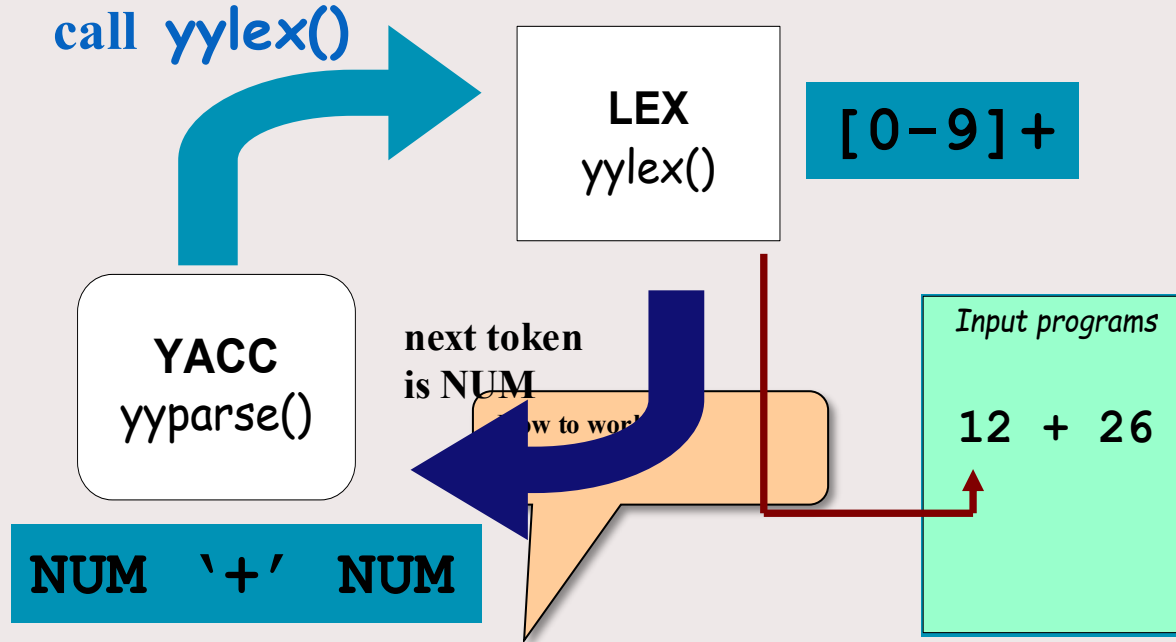
LEX allows:

- character classes, e.g. `[0-9]+` in regular expressions
- “semantic” actions with regular expression
 - `[0-9]+ { value = atoi(yytext); sum += value; }`
 - `yytext` contains the recognized *string (token)*
 - in accepting state (i.e. when regex is matched) the semantic action is executed

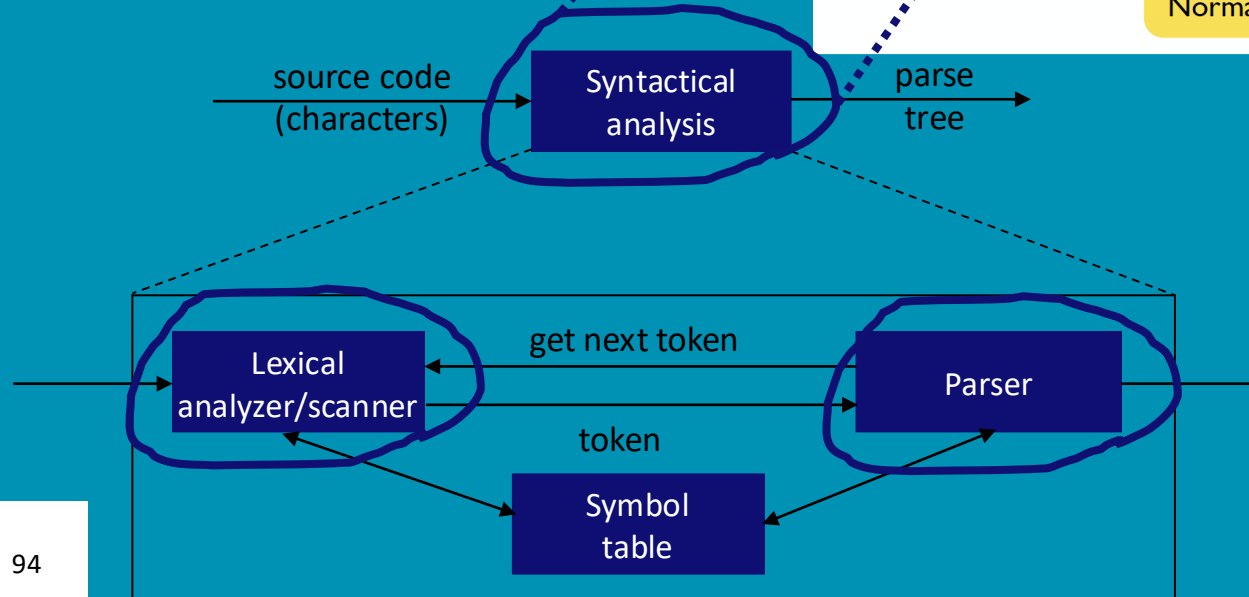
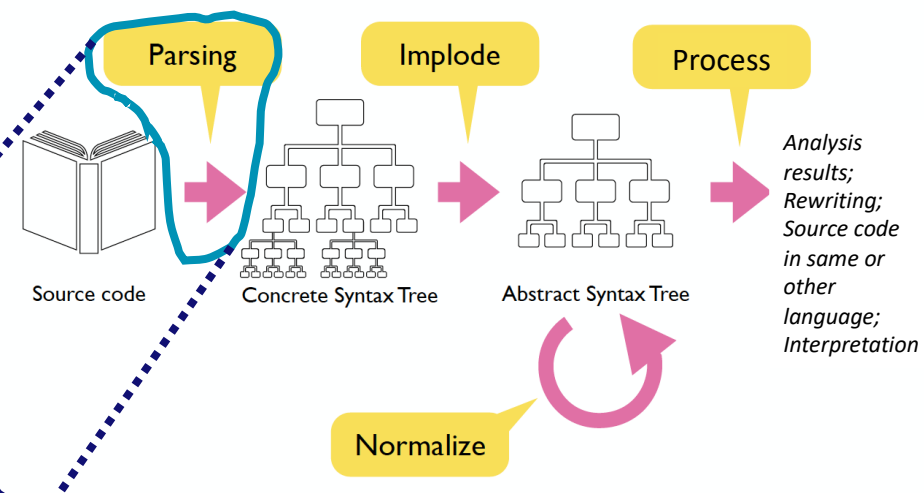
Example of a (F)LEX specification

```
/** Definition section */
%{ /* C code to be copied verbatim */
    #include <stdio.h>
%}
/* This tells flex to read only one input file */
%option noyywrap
%%
/** Rules section */
/* [0-9]+ matches a string of one or more digits */
[0-9]+ { /* yytext is a string containing the matched text. */
    printf("Saw an integer: %s\n", yytext);
}
.|\\n { /* Ignore all other characters. */ }
%%
/** C Code section */
int main(void) {
    /* Call the lexer, then quit. */
    yylex(); return 0; }
```

LEX and YACC



Recall: Scanning/lexical analysis + parsing/syntactical analysis



Syntactical analysis using YACC

gram.y

File containing desired
grammar in YACC format

yacc

YACC program

y.tab.c

C source program created by YACC

cc
or gcc

C compiler

a.out

Executable program that will parse
programs conforming to grammar given in **gram.y**

Syntactical analysis using YACC – .y File Structure

Definitions

`%%`

Rules

`%%`

Supplementary Code

*The identical LEX
format was taken from
this...*

Syntactical analysis using YACC

Rules section: a context free grammar, e.g.:

```
expr    : expr '+' term
        | term
        ;
term     : term '*' factor
        | factor
        ;
factor  : '(' expr ')'
        | ID
        | NUM
        ;
```

Syntactical analysis using YACC

Definition section

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
%token ID NUM  
%start expr
```

Terminal

Start symbol
(non-terminal)

Syntactical analysis using YACC

Some details

- LEX produces a function called *yylex()* — *scanner/tokenizer*
- YACC produces a function called *yyparse()* — *parser*
- *yyparse()* expects to be able to call *yylex()*
How to get *yylex()*?
 1. If you don't want to write your own: use *lex*!
 2. Write your own!

Syntactical analysis using YACC

Semantic actions

```
expr : expr '+' term    { $$ = $1 + $3; }
    | term              { $$ = $1; }
    ;

term : term '*' factor  { $$ = $1 * $3; }
    | factor            { $$ = $1; }
    ;

factor : '(' expr ')'   { $$ = $2; }
      | ID
      | NUM
      ;
```

Syntactical analysis using YACC

Semantic actions

```
expr : expr '+' term    { $$ = $1 + $3; }  
    | term              { $$ = $1; }  
    ;  
term : term '*' factor  { $$ = $1 * $3; }  
    | factor            { $$ = $1; }  
    ;  
factor : '(' expr ')'   { $$ = $2; }  
      | ID  
      | NUM  
      ;
```


Syntactical analysis using YACC

Semantic actions

\$1



```
expr : expr '+' term    { $$ = $1 + $3; }
    | term              { $$ = $1; }
    ;

term : term '*' factor   { $$ = $1 * $3; }
    | factor            { $$ = $1; }
    ;

factor : '(' expr ')'    { $$ = $2; }
    | ID
    | NUM
    ;
```

Syntactical analysis using YACC

Semantic actions

```
expr : expr '+' term    { $$ = $1 + $3; }
    | term              { $$ = $1; }
    ;

term : term '*' factor   { $$ = $1 * $3; }
    | factor             { $$ = $1; }
    ;

factor : '(' expr ')'    { $$ = $2; }
      | ID
      | NUM
    ;
```



Syntactical analysis using YACC

Semantic actions

```
expr : expr '+' term    { $$ = $1 + $3; }
    | term              { $$ = $1; }
    ;

term : term '*' factor  { $$ = $1 * $3; }
    | factor           { $$ = $1; }
    ;

factor : '(' expr ')'   { $$ = $2; }
      | ID
      | NUM
      ;
```



Syntactical analysis using YACC – Precedence and association

Precedence/association

```
expr: expr '-' expr
     | expr '*' expr
     | expr '<' expr
     | '(' expr ')'
     ...
     ;
```

(1) $1 - 2 - 3$

(2) $1 - 2 * 3$

1. $1-2-3 = (1-2)-3$? or $1-(2-3)$?
Define '-' operator with left-association.
2. $1-2*3 = 1-(2*3)$
Define "*" operator as precedent to "-" operator

Syntactical analysis using YACC – Precedence and association

Precedence/association

```
%left '+' '-'  
%left '*' '/'  
%noassoc UMINUS
```

```
expr  : expr '+' expr      { $$ = $1 + $3; }  
      | expr '-' expr      { $$ = $1 - $3; }  
      | expr '*' expr      { $$ = $1 * $3; }  
      | expr '/' expr      { if($3==0)  
                           yyerror("divide 0");  
                           else  
                               $$ = $1 / $3;  
                           }  
      | '-' expr %prec UMINUS { $$ = -$2; }
```

Syntactical analysis using YACC – Precedence and association

Precedence/association

```
%right '='  
%left '<' '>' NE LE GE  
%left '+' '-'  
%left '*' '/'
```



highest precedence

Syntactical analysis using YACC

YACC

- Rules *may be recursive*
- Rules *may be ambiguous*
- Uses *bottom-up Shift/Reduce parsing*
 - Get a token
 - Push onto stack
 - Can it be reduced (How do we know?)
 - If yes: Reduce using a rule
 - If no: Get another token
- YACC cannot look ahead > 1 token

Syntactical analysis using YACC – Conflicts

Shift/reduce conflict

occurs when a grammar is written in such a way that a decision between shifting and reducing can not be made.

e.g.: IF-ELSE ambiguity

- To resolve conflict, YACC will choose to shift

Reduce/reduce conflict

```
start : expr | stmt
      ;
expr  : CONSTANT;
stmt  : CONSTANT;
```

- To resolve conflict, YACC will reduce using earlier grammar rule

Not good practice to rely on this → modify grammar to eliminate them

Syntactical analysis using YACC – Use left recursion

Left recursion

```
list:
    item
    | list ',' item
    ;
```

Right recursion

```
list:
    item
    | item ',' list
    ;
```

LR parsers prefer left recursion

LL parsers prefer right recursion

YACC is an LR parser

Lexical and syntactical analysis using LEX and YACC

Further reading on LEX+YACC:

<http://dinosaur.compilertools.net>

<http://flex.sourceforge.net/manual/>

http://www.delorie.com/gnu/docs/bison/bison_toc.html

Questions?



